

Open Questions & Decisions Log

Project: Aura Promptly

Version: 1.1

Date: 2026-05-14

This document tracks architectural and design questions that require further investigation or a decision before the relevant phase begins. It also records decisions already made and their outcome.

Change Log

Version	Date	Changes
1.0	2026-05-14	Initial baseline
1.1	2026-05-14	OQ-01, OQ-03, OQ-04, OQ-05, OQ-07 resolved and moved to Resolved Decisions; OQ-02 resolved; new RD-09 through RD-15 added

Open Questions

ID	Question	Impact	Phase	Owner
OQ-06	For the Azure spoke, is Azure Container Apps the right choice, or should it be Azure App Service (simpler) or AKS (overkill)?	Phase 6 complexity and cost	Phase 6	Platform Engineer

Note: All other original open questions (OQ-01 through OQ-05, OQ-07) have been resolved — see Resolved Decisions below.

Resolved Decisions

ID	Question	Decision	Date	Rationale
RD-01	Hub cloud provider	AWS	2026-05-14	Bedrock is AWS-native; IAM provides strong multi-cloud auth
RD-02	AI gateway technology	LiteLLM Proxy (self-hosted ECS Fargate)	2026-05-14	OpenAI-compatible, built-in spend tracking, open source
RD-03	Spoke connectivity model	Public HTTPS + API Gateway + WAF	2026-05-14	Lowest cost; meets security requirements; no VPN needed
RD-04	Primary Bedrock model	Claude Haiku 3	2026-05-14	Best cost/capability for PoC tasks
RD-05	CI/CD platform	GitHub Actions with OIDC	2026-05-14	Specified by user; OIDC eliminates long-lived credentials

ID	Question	Decision	Date	Rationale
RD-06	Terraform state	Terraform Cloud free tier	2026-05-14	Free for single user; built-in UI and locking
RD-07	Observability target	Two-layer: CloudWatch (primary, always-on) + OTel → Grafana Cloud (optional, <code>enable_grafana_export</code>)	2026-05-14	CloudWatch provides zero-egress baseline for regulated environments; Grafana provides visual demo layer; OTel pipeline is vendor-neutral
RD-08	Sample app framework	Python FastAPI + Streamlit	2026-05-14	Python is AI community standard; Streamlit is demo-friendly
RD-09	OQ-01: Vector store for Bedrock KB	Aurora Serverless (pgvector) — GA confirmed May 2026	2026-05-14	~90% cost reduction vs OpenSearch Serverless; scale-to-zero when idle; shared Aurora cluster with LiteLLM DB; first-class Bedrock KB support
RD-10	OQ-02: Terraform Cloud free tier workspace limit	Unlimited workspaces supported on free tier	2026-05-14	Terraform Cloud free tier has no workspace count limit; only 1 user and 500 managed resources are constrained
RD-11	OQ-03: GCP OIDC token claim structure for Cognito	Cognito federation not used for GCP — replaced by direct AWS STS federation	2026-05-14	GCP Cloud Run service accounts issue OIDC tokens from the metadata server; these are used directly with <code>AssumeRoleWithWebIdentity</code> ; no Cognito required; simpler and AWS-documented
RD-12	OQ-04: LiteLLM native Bedrock Agent support	LiteLLM natively supports Bedrock Agent via <code>bedrock/agent/{AGENT_ID}/{ALIAS_ID}</code> route — no Lambda adapter needed	2026-05-14	Confirmed from LiteLLM docs; works through proxy config YAML; supports streaming and cost tracking
RD-13	OQ-05: Grafana Cloud free tier log retention	14 days	2026-05-14	Sufficient for PoC sessions; CloudWatch is the primary log store with configurable retention
RD-14	OQ-07: LiteLLM Docker image pinning	Pin to specific <code>--stable</code> semver tag (e.g. <code>v1.83.0-stable</code>); verify with cosign in CI/CD	2026-05-14	<code>--stable</code> tags undergo 12-hour load tests; images signed with cosign from v1.83.0; <code>main-latest</code> rejected due to breaking changes

ID	Question	Decision	Date	Rationale
RD-15	GCP auth method	GCP Workload Identity OIDC → AWS STS <code>AssumeRoleWithWebIdentity</code> → IAM Sig V4	2026-05-14	Direct federation; no Cognito; single IAM authoriser handles both AWS and GCP spokes; AWS-documented pattern
RD-16	Bedrock Agent demo scenario	"Research Assistant": Agent queries KB via <code>search_knowledge_base</code> Lambda action group and synthesises a structured board summary	2026-05-14	Self-contained (uses existing KB data); realistic financial services use case; demonstrates Agent + KB integration in one demo
RD-17	KB sync wait behaviour in recreate runbook	Poll <code>get-ingestion-job</code> until <code>COMPLETE</code> before deploying spokes	2026-05-14	Prevents Azure RAG chatbot returning empty results immediately after fresh deploy; adds 1–5 minutes to recreate time; NFR-02 (<30 min) still met
RD-18	Public repo security controls	GitHub secret scanning + push protection + gitleaks on PRs + <code>sensitive = true</code> on all Terraform secret outputs + <code>os.environ/</code> references in LiteLLM config	2026-05-14	Repo is public; prevention is the only viable strategy; GitHub native scanning catches 200+ credential patterns automatically

Design Clarifications Needed Before Phase 6

Before starting Phase 6 (Azure Spoke), resolve OQ-06:

OQ-06 — Azure Container Apps vs App Service:

- Azure Container Apps (ACA) is recommended: serverless, consumption plan (\$0 when idle), supports containers natively, no VNet required for outbound HTTPS
- Azure App Service is simpler but always-on (minimum ~\$13/month on B1 tier)
- AKS is overkill for a single demo container
- **Likely decision:** ACA — consistent with the serverless-first principle. Confirm when Azure account is set up.

Financial Services Alignment Notes

Since the PoC is modelled for a financial services context, these areas should be called out in the article/video:

Area	PoC Approach	Production Consideration
Data residency	us-east-1 (single region)	Data residency requirements may mandate specific regions or in-country hosting
Observability data egress	Optional (<code>enable_grafana_export</code>); CloudWatch is always-on with zero egress	Production would disable external export; use CloudWatch + SIEM integration
Model response logging	Logged to CloudWatch (no masking)	Production would require PII masking, WORM-compliant log retention, DLP integration
Encryption	AWS-managed KMS (SSE-S3, AES-256)	Production would use customer-managed KMS keys (CMK) with key rotation

Area	PoC Approach	Production Consideration
Audit trail	CloudWatch + LiteLLM logs	Production would integrate with SIEM (Splunk, Sentinel)
Change management	GitHub Actions CI/CD	Production would add approval gates, SoD controls
Model governance	Bedrock Guardrails (basic)	Production would add model risk management framework, bias testing
Vector store	Aurora pgvector (cost-optimised)	Production would use OpenSearch Serverless or dedicated pgvector for high-QPS workloads

These are noted as "not in scope for PoC but worth mentioning" — they make excellent LinkedIn article talking points.